MURL Chess | My findings for the group poster

Akbar Aman • UIC MATH 496 • November 3, 2026

I have brought together my September and October results, solution checks, and new time-limit experiments for our November 5 research handoff.

My objective and scope

Following our October 31 discussion and Min's email, I investigated all three agreed questions using the same 20 mate-in-three problems. I am sharing these findings so we can compare our results and select the material for our poster.

I used the corrected diagrams as my primary benchmark and my September eight-board study as runtime context. I scored a first move as correct when it matched the unique key that forces mate against every legal defense within five plies. I checked the full mating strategies separately.

1. Execution time

How do Minimax and Monte Carlo Tree Search (MCTS) compare in terms of execution time?

In my October runs, three-ply Minimax completed 20/20 searches, with a median of 0.535 s. MCTS completed 60/60 trials at 1,000 simulations, with a median of 2.724 s. Five-ply Minimax completed only puzzles 16 and 18 under my 20 s ceiling. Their median was 8.195 s. I treat the other 18 full-depth times as unknown.

I requested different workloads in these settings. In my September study, two 16-piece boards had 15 versus 53 legal first moves, visited 2,978 versus 123,342 nodes at depth three, and took 0.087 versus 3.586 s at the median. I found that piece count alone did not describe search effort.

2. Move accuracy

Which algorithm produces more accurate or optimal moves across different chess positions?

I recorded 6/20 key matches for three-ply Minimax (30%) and 9/60 for MCTS (15%) across seeds 0, 1, and 2 on the same boards. MCTS's hits covered five distinct puzzles. Both completed five-ply Minimax searches matched the key. I recorded eight further key matches at timeout, separately from completed searches.

On November 3, I tested shared ceilings. At 0.25 s, Minimax returned 4/20 keys and MCTS 3/60. At 1.00 s, they returned 5/20 and 6/60. Minimax completed 4 and 15 calls respectively. Every MCTS call stopped before its 100,000-iteration target. Median completed simulations rose from 90 to 341. Here I count the decision available at timeout.

3. Incorrect moves and algorithm limitations

What causes chess AI algorithms to choose incorrect moves, and how do search depth, position complexity, and computational limits affect those mistakes?

In puzzle 6, I recorded Qxe6 for all three October MCTS seeds. I verified that Black can reply ...Rd8, leaving White without a forced mate in the remaining three plies. The key is Qc8+. Against the same ...Rd8 defense, White has Qxd8+ Re8 Qxe8#.

At seed 0, I recorded 88 visits to Qxe6 and 56 to Qc8+. My implementation selects the most visited root move. Of 1,000 rollouts, 881 reached the material cutoff. These records explain the final selection under my policy. I have not isolated how much a different rollout policy or evaluator would improve accuracy.

Puzzle 16 gave me a direct depth comparison with the same evaluator. My three-ply Minimax run chose Kg4. After ...Kg8, White cannot force mate within the remaining horizon. My five-ply run completed in 10.012 s and chose Rf7. A printed continuation is ...Kg8, Rd7 g5, b8=Q#.

In puzzle 13, my three-ply Minimax run and all MCTS seeds chose Kxd1. I verified that ...Kg8 prevents the required short mate. The correct key is Nf6. I have included annotated boards and the checked defensive witnesses in my poster handoff.

How I checked the solutions and limits

I replayed all 20 printed main lines. Each was legal, agreed with the verified first move, and ended in checkmate. In puzzle 18, after Rxg7 Kf8 Ne6+ Ke8, I found that both c8=R# and c8=Q# mate. The booklet's queen-stalemate explanation is incorrect on this board. This does not change my first-move scores.

I tested selected tactical positions, with one Black-to-move case. My exact verifier uses a different search procedure but shares python-chess with both algorithms. I used three MCTS seeds and one November run per ceiling and seed. Machine load can change timeout decisions. I can demonstrate specific depth and budget effects here, while the cause of every missed move remains unresolved.

What I am sharing with the group

I have placed my original September 23 and October 14 PDFs, my comprehensive November report, and editable LaTeX in Reports. My poster handoff includes figures, captions, per-position results, raw calls, FENs, seeds, software versions, and reproduction commands.

Repository

https://github.com/ak23bar/murl-chess

Reports

https://github.com/ak23bar/murl-chess/tree/main/Reports

Poster figures and evidence

https://github.com/ak23bar/murl-chess/tree/main/deliverables/november05

I suggest puzzle 6 for the clearest wrong-move comparison and puzzle 16 for the depth example. We can combine these with everyone else's findings when deciding what to include. I am also happy to help integrate the selected charts, boards, and results into the LaTeX poster.